

TRABAJO PRÁCTICO 5: Optimización y Procesamiento de Datos

Estudiante: Damian Tristant

Materia: Base de Datos 2

Actividad 1)

a) Índice Simple: El campo que necesita un índice simple es `numero_seguimiento` porque el enunciado dice que es único para cada paquete. Al ponerle la propiedad de único, nos aseguramos de que no se repitan códigos y las búsquedas directas sean rapidísimas.

```
db.envios.createIndex({ numero_seguimiento: 1 }, {  
  unique: true })
```

b) Índice Compuesto: Para buscar por estado y ciudad_destino al mismo tiempo, lo ideal es un índice compuesto que junte ambos campos. Esto optimiza un montón las consultas combinadas de la empresa.

```
db.envios.createIndex({ estado: 1, ciudad_destino: 1 })
```

c) Qué pasa internamente sin índices: Si la colección es gigante y no tiene índices, MongoDB no sabe dónde está parado. Para encontrar un dato tiene que hacer un escaneo completo, registro por registro, desde el primero hasta el último (esto se conoce como **COLLSCAN**). En una base de datos grande, esto es un desastre porque consume muchísima memoria, hace que las consultas demoren una eternidad y te plancha el servidor.

2)

a) y b) Creación de la base de datos `logistica_db` e inserción de los 10 envíos:

```
use logistica_db
```

```
db.envios.insertMany([
```

```
  { numero_seguimiento: "TRK-1001", estado: "En  
Camino", ciudad_destino: "CABA", peso: 12.5 },
```

```
  { numero_seguimiento: "TRK-1002", estado:  
"Entregado", ciudad_destino: "Rosario", peso: 25.0 },
```

```
{ numero_seguimiento: "TRK-1003", estado:
"Entregado", ciudad_destino: "CABA", peso: 8.2 },

{ numero_seguimiento: "TRK-1004", estado:
"Retenido", ciudad_destino: "Córdoba", peso: 32.1 },

{ numero_seguimiento: "TRK-1005", estado: "En
Camino", ciudad_destino: "Mendoza", peso: 5.4 },

{ numero_seguimiento: "TRK-1006", estado:
"Entregado", ciudad_destino: "Rosario", peso: 18.0 },

{ numero_seguimiento: "TRK-1007", estado: "En
Camino", ciudad_destino: "Córdoba", peso: 45.0 },

{ numero_seguimiento: "TRK-1008", estado:
"Entregado", ciudad_destino: "CABA", peso: 21.5 },

{ numero_seguimiento: "TRK-1009", estado:
"Retenido", ciudad_destino: "Mendoza", peso: 14.2 },

{ numero_seguimiento: "TRK-1010", estado:
"Entregado", ciudad_destino: "Córdoba", peso: 19.8 }

])
```

```
>_MONGOSH
```

```
> use logistica_db
```

```
< switched to db logistica_db
```

```
> db.envios.insertMany([
```

```
  { numero_seguimiento: "TRK-1001", estado: "En Camino", ciudad_destino: "CABA", peso:
```

```
  { numero_seguimiento: "TRK-1002", estado: "Entregado", ciudad_destino: "Rosario", pes
```

```
  { numero_seguimiento: "TRK-1003", estado: "Entregado", ciudad_destino: "CABA", peso:
```

```
  { numero_seguimiento: "TRK-1004", estado: "Retenido", ciudad_destino: "Córdoba", peso
```

```
  { numero_seguimiento: "TRK-1005", estado: "En Camino", ciudad_destino: "Mendoza", pes
```

```
  { numero_seguimiento: "TRK-1006", estado: "Entregado", ciudad_destino: "Rosario", pes
```

```
  { numero_seguimiento: "TRK-1007", estado: "En Camino", ciudad_destino: "Córdoba", pes
```

```
  { numero_seguimiento: "TRK-1008", estado: "Entregado", ciudad_destino: "CABA", peso:
```

```
  { numero_seguimiento: "TRK-1009", estado: "Retenido", ciudad_destino: "Mendoza", peso
```

```
  { numero_seguimiento: "TRK-1010", estado: "Entregado", ciudad_destino: "Córdoba", pes
```

```
])
```

```
< {
```

```
  acknowledged: true,
```

```
  insertedIds: {
```

```
    '0': ObjectId('6a15f81ed4ecb3f875c2de8e'),
```

```
    '1': ObjectId('6a15f81ed4ecb3f875c2de8f'),
```

```
    '2': ObjectId('6a15f81ed4ecb3f875c2de90'),
```

```
    '3': ObjectId('6a15f81ed4ecb3f875c2de91'),
```

```
    '4': ObjectId('6a15f81ed4ecb3f875c2de92'),
```

```
    '5': ObjectId('6a15f81ed4ecb3f875c2de93'),
```

```
    '6': ObjectId('6a15f81ed4ecb3f875c2de94'),
```

```
    '7': ObjectId('6a15f81ed4ecb3f875c2de95'),
```

```
    '8': ObjectId('6a15f81ed4ecb3f875c2de96'),
```

```
    '9': ObjectId('6a15f81ed4ecb3f875c2de97')
```

```
  }
```

```
}
```

```
Atlas atlas-5pkm3b-shard-0 [primary] logistica_db>
```

```
}  
> db.envios.createIndex({ numero_seguimiento: 1 }, { unique: true })  
    db.envios.createIndex({ estado: 1, ciudad_destino: 1 })  
< estado_1_ciudad_destino_1  
Atlas atlas-5pkm3b-shard-0 [primary] logistica_db> |
```

Actividad 2)

1)

a) Comando de diagnóstico: Para ver si una consulta hace un escaneo completo o usa índices se usa el comando `.explain()`, pasándole el parámetro "executionStats". Ahí te tira todo el detalle técnico.

b) executionTimeMillis: Es el tiempo exacto que tardó en completarse la consulta, medido en milisegundos. Lo ideal es que dé 0 o lo más cercano posible.

totalDocsExamined: Es la cantidad total de documentos que el motor tuvo que revisar físicamente en el disco para darte el resultado. Si hay índice, este número baja un montón.

c) Solución para alto uso de memoria: Si la memoria vuela porque los documentos son enormes, la solución recomendada es usar **Proyecciones (\$project)** en las

consultas para traer solamente los campos que necesitamos en ese momento y no arrastrar toda la estructura pesada.

2) Implementación practica:

a) b)

```
db.envios.find({ numero_seguimiento: "TRK-1005"
}).explain("executionStats")
```

```
> db.envios.find({ numero_seguimiento: "TRK-1005" }).explain("executionStats")
< {
  explainVersion: '1',
  queryPlanner: {
    namespace: 'logistica_db.envios',
    parsedQuery: {
      numero_seguimiento: {
        '$eq': 'TRK-1005'
      }
    },
    indexFilterSet: false,
    queryHash: 'D2CD172B',
    planCacheShapeHash: 'D2CD172B',
    planCacheKey: '8786C56C',
    optimizationTimeMillis: 0,
    maxIndexedOrSolutionsReached: false,
    maxIndexedAndSolutionsReached: false,
    maxScansToExplodeReached: false,
    prunedSimilarIndexes: false,
    winningPlan: {
      isCached: false,
      stage: 'EXPRESS_IXSCAN',
      keyPattern: '{ numero_seguimiento: 1 }',
      indexName: 'numero_seguimiento_1'
    },
    rejectedPlans: []
  },
  executionStats: {
    executionSuccess: true,
    nReturned: 1,
    executionTimeMillis: 1,
    totalKeysExamined: 1,
    totalDocsExamined: 1,
    executionStages: {
      isCached: false,
      stage: 'EXPRESS_IXSCAN',
      keyPattern: '{ numero_seguimiento: 1 }',
      indexName: 'numero_seguimiento_1',
```



```
},
queryShapeHash: '635E04403DB63E5A93044255CC2356AF4ACE1482C2C4A65CFDD97220F',
command: {
  find: 'envios',
  filter: {
    numero_seguimiento: 'TRK-1005'
  },
  '$db': 'logistica_db'
},
serverInfo: {
  host: 'ac-cevpdwy-shard-00-01.n7r7lo6.mongodb.net',
  port: 27017,
  version: '8.0.23',
  gitVersion: 'ccf0d81588377542f00eeecf1b1ffbc095b1eeff'
},
serverParameters: {
  internalQueryFacetBufferSizeBytes: 104857600,
  internalQueryFacetMaxOutputDocSizeBytes: 104857600,
  internalLookupStageIntermediateDocumentMaxSizeBytes: 16793600,
  internalDocumentSourceGroupMaxMemoryBytes: 104857600,
  internalQueryMaxBlockingSortMemoryUsageBytes: 33554432,
  internalQueryProhibitBlockingMergeOnMongoS: 0,
  internalQueryMaxAddToSetBytes: 104857600,
  internalDocumentSourceSetWindowFieldsMaxMemoryBytes: 104857600,
  internalQueryFrameworkControl: 'trySbeRestricted',
  internalQueryPlannerIgnoreIndexWithCollationForRegex: 1
},
ok: 1,
'$clusterTime': {
  clusterTime: Timestamp({ t: 1779825144, i: 1 }),
  signature: {
    hash: Binary.createFromBase64('RaLirJCNdTqdsnem+BMkZcJWoM=', 0),
    keyId: 7603783751214039000
  }
},
operationTime: Timestamp({ t: 1779825144, i: 1 })
}
```

Al mirar el resultado en la consola, se puede ver que en la parte de las etapas figura "stage": "IXSCAN" (Index Scan). Esto confirma que MongoDB usó el índice que creamos en la Actividad 1 y no tuvo que buscar por toda la base de datos.

c)

db.envios.stats()

```
  },
  sharded: false,
  size: 1170,
  count: 10,
  numOrphanDocs: 0,
  storageSize: 36864,
  totalIndexSize: 77824,
  totalSize: 114688,
  indexSizes: {
    _id_: 36864,
    numero_seguimiento_1: 20480,
    estado_1_ciudad_destino_1: 20480
  },
  avgObjSize: 117,
  ns: 'logistica_db.envios',
  nindexes: 3,
  scaleFactor: 1
}
```

Actividad 3:

1)

a) Es una herramienta para armar reportes y procesar datos de forma avanzada. Funciona como una "**tubería**" porque los documentos van pasando por distintas etapas en orden. Lo que sale filtrado o transformado de una etapa entra directamente a la que sigue, permitiendo hacer cálculos complejos de un tirón.

b) \$match: Filtra los documentos para dejar pasar solo los que cumplen una condición (como el WHERE de SQL).

\$group: Agrupa los datos según una clave y te deja aplicar funciones como sumar o contar.

\$project: Sirve para darle forma al resultado final, eligiendo qué campos mostrar, cuáles ocultar o calculando campos nuevos.

c) Para hacer un join (unión) con otra colección (como clientes) se usa el operador \$lookup.

2) Implementación practica:

a)

```
db.envios.aggregate([  
  { $match: { estado: "Entregado" } },  
  { $group: { _id: "$ciudad_destino", total_entregados: {  
$sum: 1 } } }  
])
```

```
> db.envios.aggregate([  
  { $match: { estado: "Entregado" } },  
  { $group: { _id: "$ciudad_destino", total_entregados: { $sum: 1 } } }  
])  
< {  
  _id: 'Córdoba',  
  total_entregados: 1  
}  
{  
  _id: 'Rosario',  
  total_entregados: 2  
}  
{  
  _id: 'CABA',  
  total_entregados: 2  
}
```

b)

```
db.envios.aggregate([
```

```
    { $group: { _id: null, peso_total_acumulado: { $sum: "$peso" } } }  
  ])
```

```
> db.envios.aggregate([  
  { $group: { _id: null, peso_total_acumulado: { $sum: "$peso" } } }  
])  
< {  
  _id: null,  
  peso_total_acumulado: 201.7  
}
```

c)

```
db.envios.aggregate([  
  { $project: { _id: 0, numero_seguimiento: { $toUpper: "$numero_seguimiento" }, estado: 1 } }  
])
```

```
< {  
  estado: 'En Camino',  
  numero_seguimiento: 'TRK-1001'  
}  
{  
  estado: 'Entregado',  
  numero_seguimiento: 'TRK-1002'  
}  
{  
  estado: 'Entregado',  
  numero_seguimiento: 'TRK-1003'  
}  
{  
  estado: 'Retenido',  
  numero_seguimiento: 'TRK-1004'  
}  
{  
  estado: 'En Camino',  
  numero_seguimiento: 'TRK-1005'  
}  
{  
  estado: 'Entregado',  
  numero_seguimiento: 'TRK-1006'  
}  
{  
  estado: 'En Camino',  
  numero_seguimiento: 'TRK-1007'  
}  
{  
  estado: 'Entregado',  
  numero_seguimiento: 'TRK-1008'  
}  
{  
  estado: 'Retenido',  
  numero_seguimiento: 'TRK-1009'  
}  
{
```

d)

```
db.envios.aggregate([
```

```
  { $project: {
```

```
    numero_seguimiento: 1,
```

```
    peso: 1,
```

```
    tipo_carga: { $cond: { if: { $gt: ["$peso", 20] },
```

```
    then: "Pesada", else: "Ligera" } }
```

```
  }
```

```
}
```

```
])
```

```

        numero_seguimiento: 1,
        peso: 1,
        tipo_carga: { $cond: { if: { $gt: ["$peso", 20] }, then: "Pesada", else: "Ligera"
      }
    }
  })
< {
  _id: ObjectId('6a15f81ed4ecb3f875c2de8e'),
  numero_seguimiento: 'TRK-1001',
  peso: 12.5,
  tipo_carga: 'Ligera'
}
{
  _id: ObjectId('6a15f81ed4ecb3f875c2de8f'),
  numero_seguimiento: 'TRK-1002',
  peso: 25,
  tipo_carga: 'Pesada'
}
{
  _id: ObjectId('6a15f81ed4ecb3f875c2de90'),
  numero_seguimiento: 'TRK-1003',
  peso: 8.2,
  tipo_carga: 'Ligera'
}
{
  _id: ObjectId('6a15f81ed4ecb3f875c2de91'),
  numero_seguimiento: 'TRK-1004',
  peso: 32.1,
  tipo_carga: 'Pesada'
}
{
  _id: ObjectId('6a15f81ed4ecb3f875c2de92'),
  numero_seguimiento: 'TRK-1005',
  peso: 5.4,
  tipo_carga: 'Ligera'
}
{

```


